Assignment 2: Predicting GitHub User Activity

Ella Tso · Student ID 113700058

Code: github.com/ellatso/NYCU-BigData-Assignment2

Big Data Analytics Techniques and Applications

Fall 2026

Overview

LightGBM on time-aware relational features built from rows visible at prediction time

- **Task** for each of 4,465 test users, predict whether they open_pr, open_issue, issue_comment, pr_review, pr_review_comment, push during 11/22–11/29 (Taiwan).
- **Data** GH Archive hourly files → official preprocess.py → 8 relational tables (9,000-user cohort, anonymized).
- **Method** weekly as-of samples → temporal + relational (FK-join) features → 6 LightGBM heads → per-activity thresholds.
- **Evaluation** Score = (macro-F1 + micro-F1) / 2 on later weeks; train / calibration / development split by time; 11 ablations vs. copy-last-week.
- **Code** github.com/ellatso/NYCU-BigData-Assignment2

1,608
hourly GH Archive files 09/01–11/07 UTC

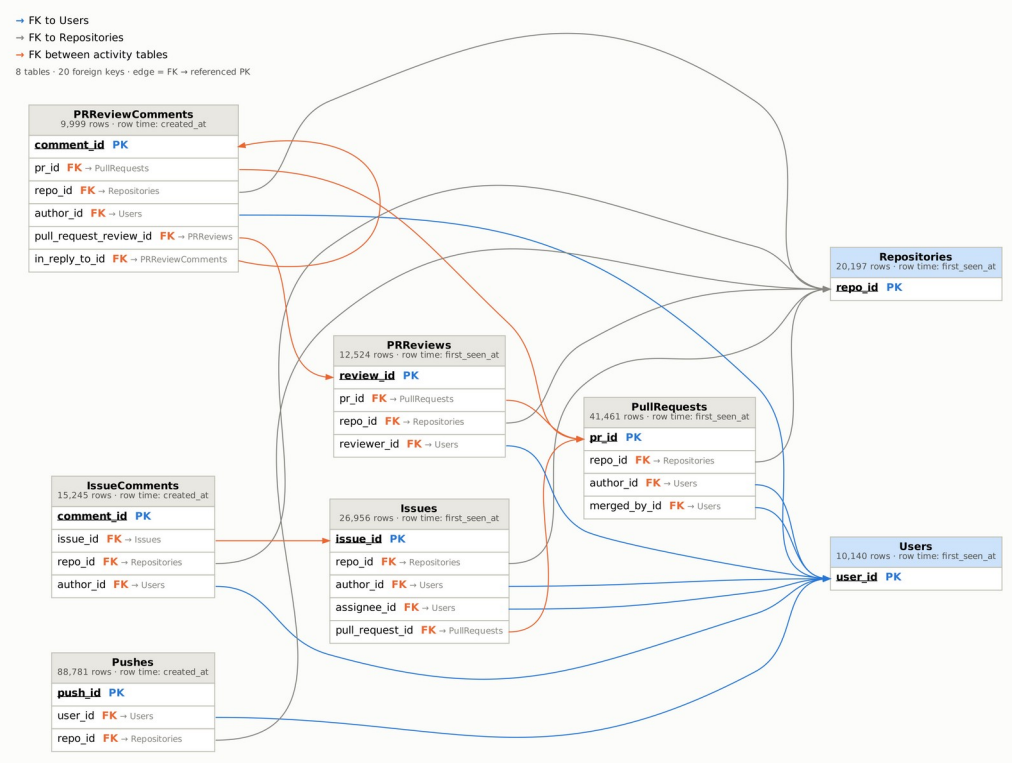
11
matched ablations with bootstrap 95% CIs

+1.6 pt
best temporal variants vs. copy-last-week (CI excludes 0)

Identical
CSV after an independent rebuild + refit

(1%) Question 1 Schema graph

8 tables as nodes and 20 foreign keys as edges (FK → referenced PK)

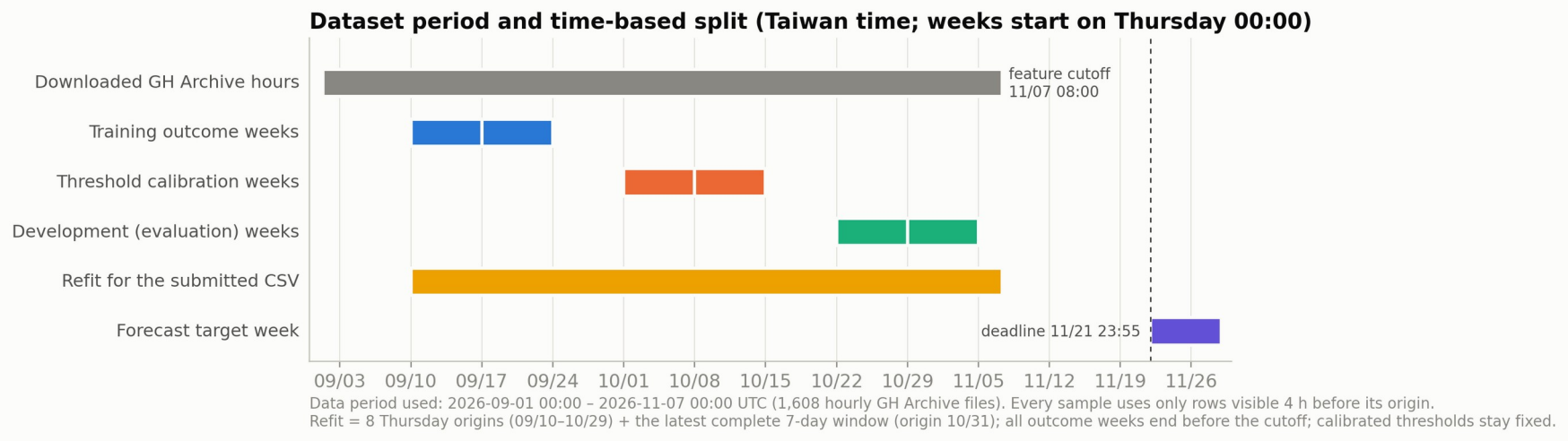


How to read it

- PK (blue) is the row number 0…n−1; an FK (orange) stores the referenced row number.
- 20 FKs: 8 → Users, 6 → Repositories, 6 between activity tables, incl. the self-reference PRReviewComments.in_reply_to_id.
- Each node shows its rows in our build and its row time — the column that decides when a row becomes visible (used for as-of filtering, Q3).
- Issues.pull_request_id links a PR conversation to its PR; PRReviewComments link to both PRReviews and PullRequests.

(1%) Question 1 Dataset time period

Data period used: 2026-09-01 00:00 – 2026-11-07 00:00 UTC (1,608 hourly GH Archive files)



1,608 h
all hourly files downloaded & gzip-verified (28.7 GB)

8 tables
official preprocess.py; PK/FK checks pass

9 origins
final refit 09/10–10/31 (Taiwan, outcome ≤ cutoff)

352 h
lead from the feature cutoff to the target week

(3%) Question 2 Relational features from FK joins

Time-windowed FK aggregations turn the 8-table graph into user-level signals

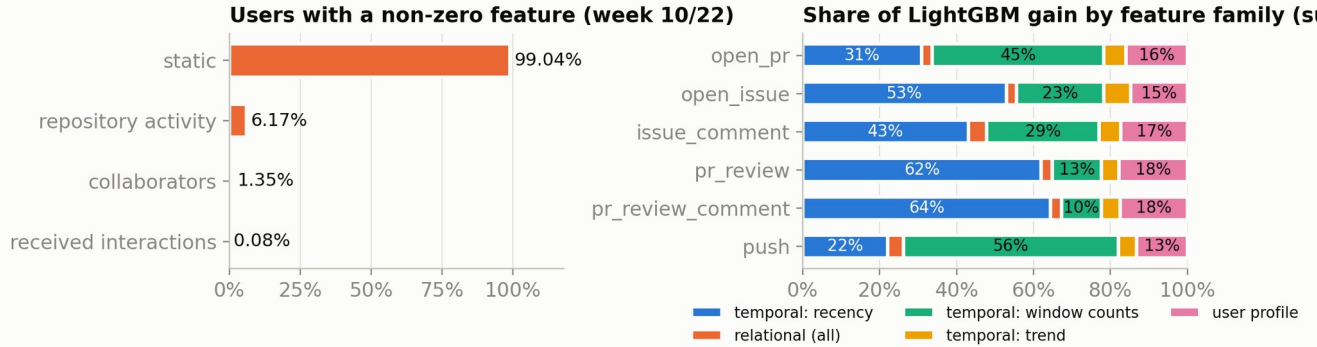
Family	FK path	What is computed	#
Repository activity	user → events → Repositories ← events ← other users	Others' activity per type in repos the user touched (90 d); 7-day mean / max; own events removed	12
Reviewed-repo PRs	user → PRReviews → Repositories ← PullRequests	PRs opened by others in the last 3 d in repos the user reviewed (90 d)	1
Received interactions	user ← PR / Issue.author ← IssueComments, PRReviews, PRReviewComments	Replies received on the user's own PRs / issues in 3 and 7 d, and distinct partners (self excluded)	12
Collaborators	user ↔ co-actor on the same PR / issue (cohort only)	Collaborators' 7-day activity per type; number of collaborators (90 d)	7
Static baseline	user → Repositories ← all users (all history)	Degree, concentration, neighbours' activity, all-time replies received	18

Design choices short windows (3 / 7 d) capture current collaboration; 'others' always excludes the user; collaborator activity uses cohort users only, because non-cohort users' activity is incomplete by construction. Submitted model: 7 user + 36 temporal + 32 dynamic relational = 75 inputs.

(3%) Question 2 What the relational signal looks like

The tables keep only cohort users' own events, so replies from others are rarely observed

Why relations add little here: the cohort-only data rarely contains other people's replies



40 / 10,524
comments on cohort-authored threads written by someone else

0.08%
of users have any non-zero received-interaction feature

3–5%
of the submitted model's gain comes from relational features

(3%) Question 2 Relational ablation

Relations do not raise the development Score on top of temporal features

Variant (development weeks 10/22 + 10/29)	Dev Score	Δ vs temporal (95% CI)
Temporal only (7/14/30 d)	35.73%	—
+ static relations	35.02%	−0.71 pt (−1.36, −0.05)
+ received interactions	35.73%	0.00 (features all zero)
+ dynamic relations (submitted)	**35.12%**	**−0.60 pt (−1.17, −0.06)**
dynamic relations − collaborators	35.30%	−0.43 pt (−0.97, +0.15)
Static relations only (no temporal)	33.45%	−2.27 pt (−3.41, −1.13)

Paired user-level bootstrap, 1,000 resamples within each week (10,334 user-weeks). On the TA reference week 09/27–10/04 (higher-activity regime), the submitted model is 53.95% vs 53.79% for temporal only.

Why the relational model is submitted

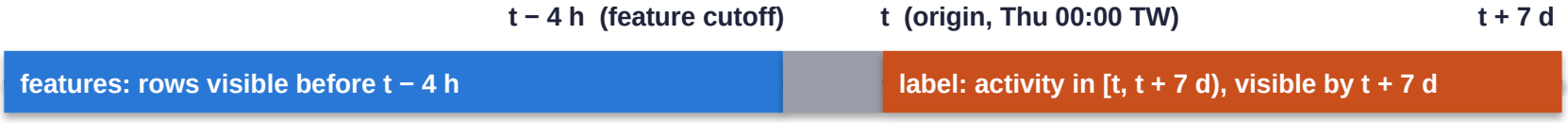
- It was fixed in the config before any real-data run, so the choice is not tuned on development weeks.
- Its gap to temporal-only is < 1 pt, and it stays above copy-last-week (+0.30 pt).

What would make relations useful

- Observing non-cohort replies, or learning over the graph end-to-end: a heterogeneous temporal GNN / RDL (TA GNN with time encoding: 54.82% on the reference week).

(5%) Question 3 As-of sampling prevents leakage

Every training row is rebuilt exactly as the archive looked at prediction time t



Row time, not creation time

A row enters only after its row time (first_seen_at / created_at); PRs, issues and reviews can appear weeks after created_at.

Visible endpoints

The event must occur before the cutoff and its user and repository rows must already be visible.

No later-known columns

merged_at, merged_by_id, closed_at, state_reason and dismissed_at are never used.

Time-ordered splits

Weekly origins; train 09/10–09/17, calibration 10/01–10/08, development 10/22–10/29; each split's labels end before the next split's cutoff.

(5%) Question 3 Temporal features

Multi-window counts, recency and trend for each activity; recency carries most of the signal

Group	Definition (per user, at cutoff t − 4 h)	#
Window counts	log(1 + count) of each activity in the last 7 / 14 / 30 d	18
Activity span	active days and distinct repositories in 7 / 14 / 30 d	6
Recency	days since the last occurrence of each activity (cap 400)	6
Trend	$(count_7 + 1) / (count_{30} / 4 + 1)$ for each activity	6
User profile	days since first seen; log lifetime count of each activity	7

Counts are log-scaled; windows stop at the cutoff, so a 30-day window at an early origin only sees collected days (warm-up limits are logged in coverage.csv).

62–64%

of LightGBM gain from recency for pr_review and pr_review_comment

56%

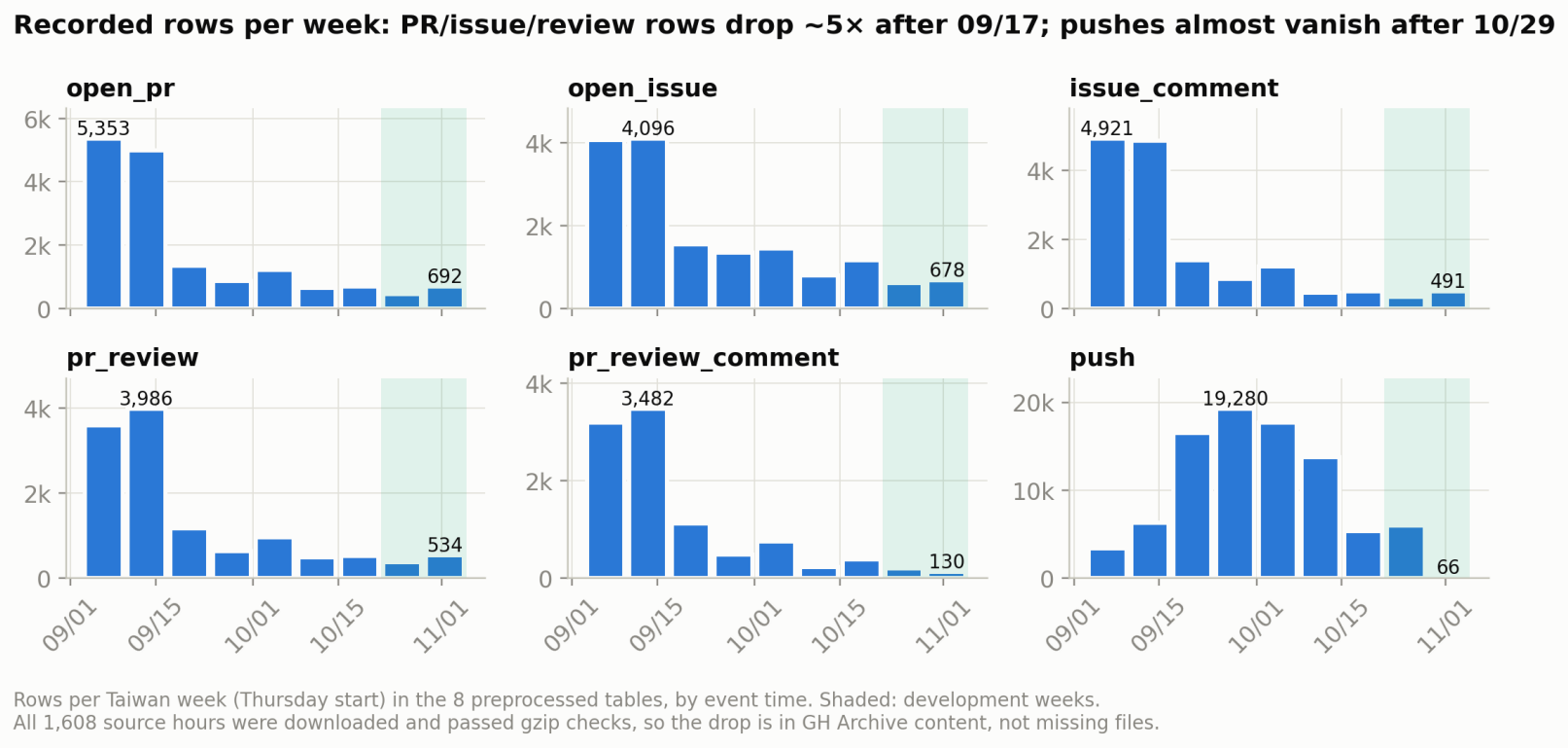
of gain from window counts for push (45% for open_pr)

Top 5 inputs

recency of pr_review, account age, recency of pr_review_comment, open_issue, issue_comment

(5%) Question 3 The data are not stationary

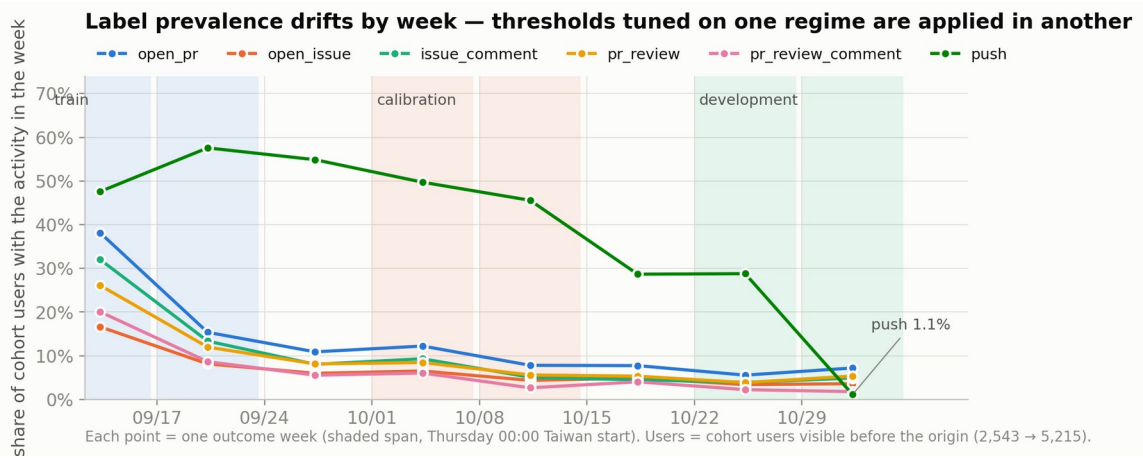
Recorded rows fall ~5× after 09/17 and pushes nearly vanish after 10/29



Rows per Taiwan week (Thursday start) in the 8 preprocessed tables, by event time. Shaded: development weeks.
All 1,608 source hours were downloaded and passed gzip checks, so the drop is in GH Archive content, not missing files.

(5%) Question 3 Label drift and threshold calibration

Positive rates shift by week, so thresholds are tuned per activity on separate calibration weeks



Thresholds

- Chosen to maximise the mean weekly Score on 10/01 + 10/08, then frozen: open_pr .30 · open_issue .275 · issue_comment .375 · pr_review .175 · pr_review_comment .225 · push .425.

Regime shift

- Calibration Score ≈ 52% vs development ≈ 35%.
- Push prevalence 28.8% → 1.1%: the six-activity Score drops from 41–45% (week 10/22) to 21–24% (10/29) for every variant, while the five-activity Score without push stays at 31–36%.

(5%) Question 3 Visibility time: the availability clock

Issues often enter the archive long after they were opened; counting by visibility time helps

Table	Rows	Visible > 24 h after event time
Issues (open_issue)	19,379	30.4%
PullRequests (open_pr)	16,694	2.8%
PRReviews (pr_review)	12,524	0.4%
IssueComments, PRReviewComments, Pushes	114,025	0%

Occurrence clock

windows ask “what happened in the last k days?” — an old issue that only just became visible is counted in its original week.

Availability clock

windows ask “what newly became visible in the last k days?” — the view the archive actually offers at prediction time.

36.41%

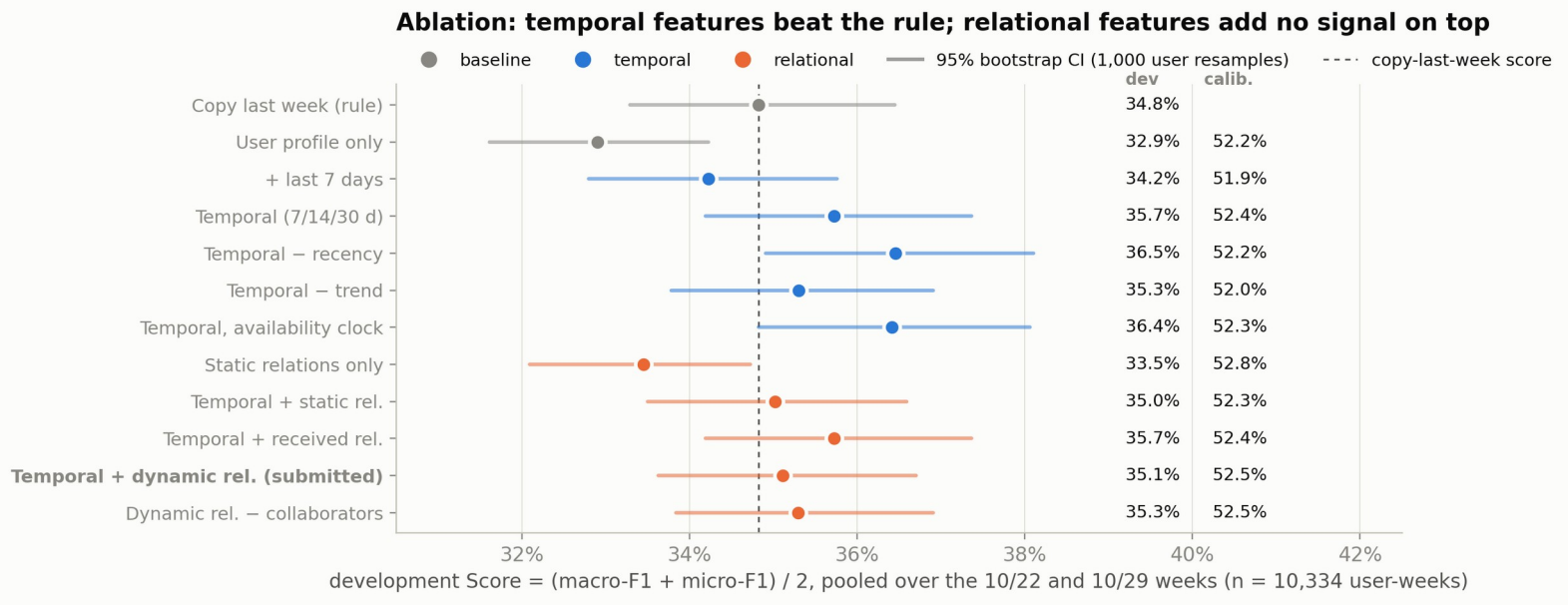
development Score with the availability clock (best temporal variants: 36.4–36.5%)

+1.59 pt

vs copy-last-week;
95% CI +0.62 to +2.62 pt

(5%) Question 3 Ablation with bootstrap CIs

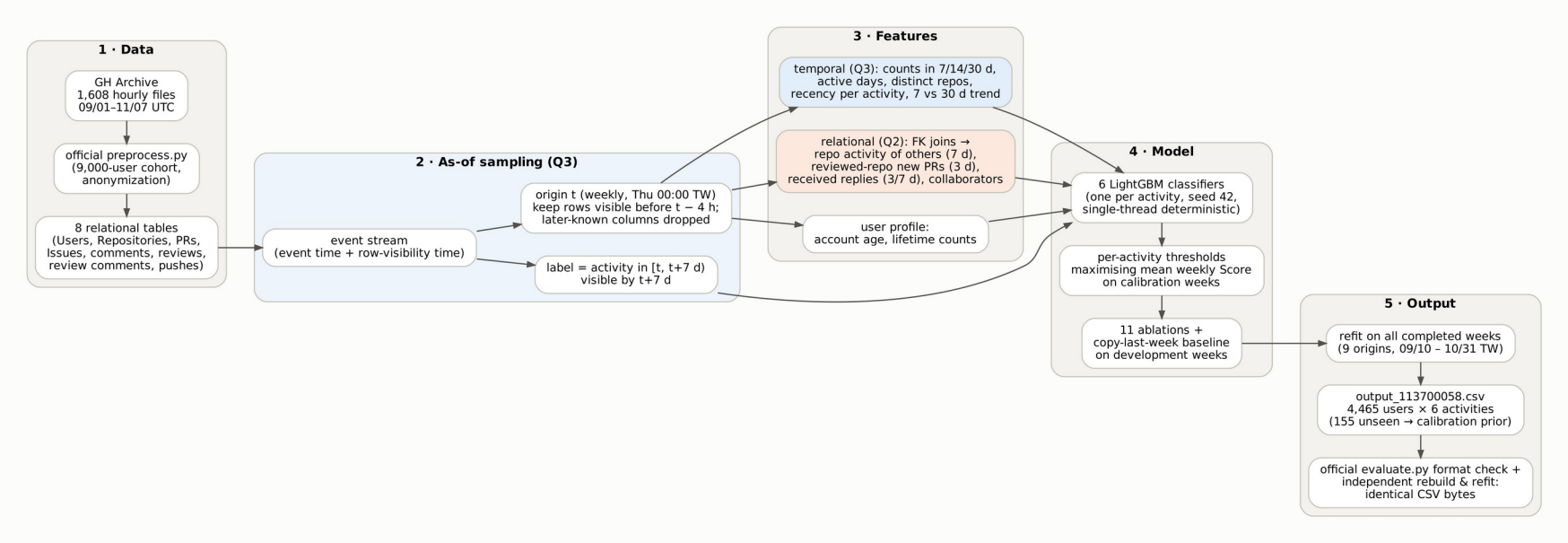
Temporal information is what beats copy-last-week; all variants share dates, users, labels, tuning



Δ vs copy-last-week: user profile only −1.89 pt (CI excludes 0) · + 7/14/30-day windows, recency, trend +0.91 pt (P(better) = 0.97) · removing recency +1.64 pt and availability clock +1.59 pt (both CIs exclude 0). Dev. weeks 10/22 + 10/29, 10,334 user-weeks, 1,000 paired user resamples.

(5%) Question 4 Framework

From GH Archive hours to a checked, reproducible submission file



Blue = temporal (Q3), orange = relational (Q2). The same as-of sampler produces training, calibration, development and test rows, so features at prediction time are computed exactly as in training.

(5%) Question 4 Pipeline steps and evaluation

Six steps, each with a check; our labels reproduce the TA's copy-last-week reference

1. **Download** 1,608 hourly files; SHA-256 recorded; gzip verified; resumable.
2. **Build tables** unchanged official preprocess.py → 8 tables; PK/FK checks.
3. **Sample** weekly origins (Thu 00:00 TW); as-of features + 7-day labels.
4. **Train** 6 LightGBM heads: 160 trees, lr 0.045, 23 leaves, seed 42, deterministic.
5. **Calibrate + evaluate** thresholds on 10/01 + 10/08; 11 ablations on 10/22 + 10/29.
6. **Refit + predict** 9 origins 09/10–10/31; 4,465 users; 155 unseen → calibration prior.

Reference week 09/27–10/04	TA README	Ours
Copy last week	49.61%	50.39%
GNN with time encoding	54.82%	—
Temporal (7/14/30 d)	—	53.79%
Temporal + dynamic rel. (submitted)	—	53.95%

- Our copy-last-week score is within 0.8 pt of the README value, so our label construction is consistent with the TA's.
- Our model rows are optimistic: this week overlaps the calibration window.

(5%) Question 4 Output and conclusions

The submitted CSV can be regenerated byte for byte from GH Archive with the code on GitHub

4,465 × 6

rows in input order; official evaluate.py format check passes

Identical CSV

after rebuilding all 8 tables and refitting (SHA-256 f5be6efe…)

Identical model

checkpoint re-trained on a second machine

Conclusions

- Time is the main signal: multi-window counts and recency beat copy-last-week; visibility-aware counting helps most under drift.
- Relational aggregation adds little here because cohort-only tables rarely contain other users' replies (40 of 10,524 comments).
- Strong distribution shift (volume −5×, push → 1%) dominates the scores; per-activity thresholds on separate weeks limit the damage.

Limitations and next steps

- Two development weeks: most differences are within ±1.5 pt and CIs overlap.
- Next: a heterogeneous temporal GNN / RDL model and drift-aware threshold updates.

Code and results: github.com/ellatso/NYCU-BigData-Assignment2